Aplicação de Design Patterns

Desenvolver uma aplicação orientada a objetos em Java puro aplicando os conceitos de Design Patterns (Padrões de Projeto) para resolver problemas reais de arquitetura e design de software. O projeto deve demonstrar a utilização prática de 6 padrões de projeto, sendo 2 de cada categoria GoF (Criacionais, Estruturais e Comportamentais).

Regras:

1. **Exclusividade de Tema:**
   - Cada grupo deverá escolher um domínio de negócio real único (ex.: Sistema de Reservas de Hotel, Plataforma de Streaming de Áudio, Sistema de Gestão de Biblioteca, Motor de Cotação de Seguros, Automação Residencial, etc.).
   - Envio de Tema via Blackboard: O tema escolhido deve ser submetido no fórum/tópico de validação de temas no Blackboard.
   - Critério de Prioridade: A definição e reserva do tema serão garantidas estritamente pela ordem de envio. Caso um tema seja repetido, o segundo grupo a enviar deverá submeter uma nova proposta.

2. **Limite de Reutilização de Exemplo:**
   - Para garantir o aprendizado individual e a aplicação prática em novos cenários, os alunos poderão utilizar no máximo 2 (dois) exemplos/padrões dentre os que foram ministrados e demonstrados em sala pelo professor.
   - Os outros 4 (quatro) padrões aplicados devem ser investigados e implementados dentro do contexto de negócio escolhido sem copiar os exemplos da aula.

3. **Proibido Exemplos Acadêmicos Abstratos:**
   - Trabalhos baseados em `Animal/Cachorro`, `Forma/Retangulo`, `Pessoa/Aluno` ou cenários genéricos desprovidos de regras de negócio reais não serão aceitos.

Requisitos:

1. Aplicação dos Padrões de Projeto

O sistema deve implementar obrigatoriamente 2 padrões de cada uma das 3 categorias GoF:

- **Padrões Criacionais** (Escolher 2):
  - Exemplos: `Factory Method`, `Abstract Factory`, `Builder`, `Prototype`, `Singleton`.
- **Padrões Estruturais** (Escolher 2):
  - Exemplos: `Adapter`, `Bridge`, `Composite`, `Decorator`, `Facade`, `Flyweight`, `Proxy`.
- **Padrões Comportamentais** (Escolher 2):
  - Exemplos: `Strategy`, `Observer`, `Chain of Responsibility`, `Command`, `State`, `Template Method`, `Iterator`, `Mediator`, `Memento`, `Visitor`.

Atenção: Cada padrão utilizado deve resolver um problema real do domínio escolhido (ex.: `Strategy` para diferentes cálculos de taxa; `Factory Method` para criação de notificações; `Decorator` para adicionar adicionais a um pedido; `Observer` para escutar mudanças de status de entrega).

2. Estrutura e Modelagem do Código

- **Linguagem**: Java puro, sem frameworks pesados de injeção de dependência (como `Spring Boot`) para demonstrar a implementação nativa dos padrões.
- **Organização e Encapsulamento**: Separação clara entre as camadas do sistema (Domínio, Padrões/Estratégias, Serviços, Main).
- **Documentação de Padrões** (Javadoc / Comentários): Cada classe que atuar como um componente de um padrão deve indicar expressamente no Javadoc a sua função (ex.: @note Papel no Padrão: Concrete Strategy, @note Papel no Padrão: Factory Method, @note Papel no Padrão: Decorator).

3. Ponto de Entrada (Main.java)

- **Interatividade Obrigatória:** O arquivo `Main.java` não deve apenas conter um fluxo pré-definido e hardcoded no código. Ele deve oferecer um menu interativo via terminal (usando `Scanner` ou similar) que permita ao usuário/avaliador interagir em tempo real com o sistema.
- **Escolha de Fluxos e Configurações:** O usuário deve ser capaz de navegar por opções, selecionar parâmetros (ex.: escolher método de pagamento, alterar estado de um pedido, selecionar tipo de notificação, adicionar adicionais/decoradores) e visualizar os Design Patterns em ação dinamicamente conforme as escolhas realizadas.
- **Visualização dos Padrões:** A execução no terminal deve deixar transparente quando um padrão está sendo acionado (ex.: mensagens de log/output indicando a criação via Factory, a execução da Strategy selecionada, a notificação dos Observers, etc.).

Instruções de Entrega

- **Repositório GitHub:**
  - Código-fonte organizado no padrão (`src/main/java`).
  - Arquivo `.gitignore` configurado adequadamente (ignorar `target/`, `build/`, `.idea/`, arquivos `.class` e `.iml`).
  - Arquivo `README.md` no repositório com uma tabela explicativa indicando:
    1. Nome do Padrão.
    2. Categoria (Criacional, Estrutural, Comportamental).
    3. Qual problema de negócio o padrão resolveu no projeto.
  - **Submissão:** Enviar o link do repositório no Blackboard na atividade correspondente.
  - **Prazo Limite:** Até às **23h59** do dia da avaliação. Commits realizados após este horário serão desconsiderados.